SQL Quick Reference — Unit 7

Every Command You Need to Know

Data Types

Type	Stores	Example
`TEXT`	Strings	`'The Dark Knight'`, `'Action'`
`INTEGER`	Whole numbers	`2008`, `42`, `-5`
`REAL`	Decimal numbers	`9.0`, `3.14`, `8.7`
`NULL`	No value / empty	(nothing)

CREATE TABLE

```
CREATE TABLE IF NOT EXISTS table_name (
    id       INTEGER PRIMARY KEY AUTOINCREMENT,
    column1  TEXT NOT NULL,
    column2  TEXT NOT NULL,
    column3  REAL,
    column4  INTEGER
);
```

Modifier	Meaning
`PRIMARY KEY`	Unique ID for each row — no duplicates
`AUTOINCREMENT`	Database assigns 1, 2, 3... automatically
`NOT NULL`	Required — INSERT fails if you skip it

Modifier	Meaning
`IF NOT EXISTS`	Won't crash if table already exists

INSERT INTO

```
-- Insert one row
INSERT INTO movies (title, genre, rating, year)
VALUES ('Inception', 'Sci-Fi', 8.8, 2010);
```

Rules:

- Column list in first `( )`, values in second `( )`
- Text uses single quotes: `'Sci-Fi'`
- Numbers have no quotes: `8.8`
- Never provide the `id` column
- Apostrophes: double them → `'Schindler''s List'`

SELECT

```
-- All rows, all columns
SELECT * FROM movies;

-- Specific columns
SELECT title, rating FROM movies;

-- Unique values only
SELECT DISTINCT genre FROM movies;
```

WHERE — Filter Rows

```sql
SELECT * FROM movies WHERE genre = 'Action';
SELECT * FROM movies WHERE rating > 8;
SELECT * FROM movies WHERE year != 2020;
SELECT * FROM movies WHERE rating >= 7.5;
```

Comparison Operators

Operator	Meaning
`=`	Equals
`!=`	Not equals
`>`	Greater than
`<`	Less than
`>=`	Greater than or equal
`<=`	Less than or equal

AND / OR — Combine Conditions

```sql
-- Both must be true
SELECT * FROM movies WHERE genre = 'Action' AND rating > 8;

-- Either can be true
SELECT * FROM movies WHERE genre = 'Action' OR genre = 'Sci-Fi';

-- Use parentheses for complex logic
SELECT * FROM movies WHERE (genre = 'Action' OR genre = 'Sci-Fi') AND rating > 8;
```

Keyword	Meaning
`AND`	Both conditions must be true
`OR`	Either condition can be true

ORDER BY — Sort Results

```sql
-- Highest first (descending)
SELECT * FROM movies ORDER BY rating DESC;

-- Lowest first / A-Z (ascending — default)
SELECT * FROM movies ORDER BY title ASC;

-- ASC is the default — these two are identical:
SELECT * FROM movies ORDER BY title;
SELECT * FROM movies ORDER BY title ASC;
```

Keyword	Direction	Example
`ASC`	Ascending (default)	A→Z, 1→10, oldest→newest
`DESC`	Descending	Z→A, 10→1, newest→oldest

LIMIT — Cap Results

```sql
-- Only return 5 rows
SELECT * FROM movies LIMIT 5;

-- Top 5 highest rated (combine with ORDER BY)
SELECT title, rating FROM movies ORDER BY rating DESC LIMIT 5;
```

LIKE — Text Search

```sql
-- Contains "dark" anywhere
SELECT * FROM movies WHERE title LIKE '%dark%';

-- Starts with "The"
SELECT * FROM movies WHERE title LIKE 'The%';

-- Ends with "man"
SELECT * FROM movies WHERE title LIKE '%man';
```

Pattern	Meaning
`'%word%'`	Contains "word" anywhere
`'word%'`	Starts with "word"
`'%word'`	Ends with "word"
`'_o%'`	Second character is "o"

Note: LIKE is case-insensitive in SQLite.

Aggregate Functions

```sql
SELECT COUNT(*) FROM movies;                -- Number of rows
SELECT AVG(rating) FROM movies;             -- Average
SELECT MAX(rating) FROM movies;             -- Highest value
SELECT MIN(year) FROM movies;               -- Lowest value
SELECT SUM(rating) FROM movies;             -- Total of all values
SELECT ROUND(AVG(rating), 1) FROM movies;   -- Round to 1 decimal
```

Function	Returns
`COUNT(*)`	Number of rows
`AVG(column)`	Mean average
`MAX(column)`	Highest value

Function	Returns
`MIN(column)`	Lowest value
`SUM(column)`	All values added together
`ROUND(value, n)`	Value rounded to n decimals

GROUP BY — Per-Category Aggregation

```sql
-- Count movies in each genre
SELECT genre, COUNT(*) FROM movies GROUP BY genre;

-- Average rating per genre, sorted
SELECT genre, ROUND(AVG(rating), 1) as avg_rating, COUNT(*) as total
FROM movies
GROUP BY genre
ORDER BY avg_rating DESC;
```

How it works: GROUP BY splits rows into groups. Aggregates (COUNT, AVG, etc.) calculate within each group. Result = one row per group.

HAVING — Filter Groups

```sql
-- Only genres with 5+ movies
SELECT genre, COUNT(*) as total FROM movies
GROUP BY genre
HAVING COUNT(*) >= 5;
```

Keyword	Filters	When
`WHERE`	Individual rows	Before grouping
`HAVING`	Groups	After GROUP BY

UPDATE — Change Existing Rows

```sql
-- Change one column
UPDATE movies SET rating = 9.5 WHERE title = 'The Dark Knight';

-- Change multiple columns
UPDATE movies SET genre = 'Sci-Fi', rating = 9.0 WHERE title = 'The Matrix';

-- Math on existing values
UPDATE movies SET rating = rating + 0.1 WHERE genre = 'Comedy';
UPDATE movies SET price = price * 0.8 WHERE brand = 'Reebok';
```

🚨 GOLDEN RULE

```sql
-- ⚠️ NEVER do this — changes ALL rows:
UPDATE movies SET rating = 0;

-- ✅ ALWAYS include WHERE:
UPDATE movies SET rating = 0 WHERE id = 5;
```

DELETE — Remove Rows

```sql
-- Delete specific rows
DELETE FROM movies WHERE rating < 3;

-- Delete by ID (safest — always unique)
DELETE FROM movies WHERE id = 42;

-- Delete by text match
DELETE FROM movies WHERE title = 'The Room';
```

🚨 GOLDEN RULE

```
-- ⚠️ NEVER do this — deletes ALL rows:
DELETE FROM movies;

-- ✅ ALWAYS include WHERE:
DELETE FROM movies WHERE id = 42;
```

DELETE vs DROP

Command	Effect	Table survives?
`DELETE FROM t WHERE ...`	Removes matching rows	✅ Yes
`DELETE FROM t`	Removes ALL rows	✅ Yes (empty)
`DROP TABLE t`	Removes entire table	❌ No

NULL Checks

```
-- Find rows where a column is empty
SELECT * FROM movies WHERE review IS NULL;

-- Find rows where a column has a value
SELECT * FROM movies WHERE review IS NOT NULL;
```

Note: Use `IS NULL` / `IS NOT NULL`, not `= NULL`.

Column Aliases — `as`

```
-- Give a column a readable name in results
SELECT genre, COUNT(*) as total FROM movies GROUP BY genre;
SELECT brand, ROUND(AVG(price), 2) as avg_price FROM sneakers GROUP BY brand;
```

The alias (`total`, `avg_price`) appears as the column header in results. You can use it in ORDER BY:

```sql
SELECT genre, COUNT(*) as total FROM movies GROUP BY genre ORDER BY total DESC;
```

Clause Order

When combining clauses, they **must** appear in this order:

```sql
SELECT columns
FROM table
WHERE condition          -- filter rows
GROUP BY column          -- group for aggregation
HAVING condition         -- filter groups
ORDER BY column ASC/DESC -- sort results
LIMIT n;                 -- cap results
```

Not every clause is required — but when used, this order is mandatory.

Quick Examples

```sql
-- Top 5 action movies by rating
SELECT title, rating FROM movies
WHERE genre = 'Action'
ORDER BY rating DESC
LIMIT 5;

-- Genre stats for genres with 3+ movies
SELECT genre, COUNT(*) as total, ROUND(AVG(rating), 1) as avg
FROM movies
GROUP BY genre
HAVING COUNT(*) >= 3
ORDER BY avg DESC;

-- Search and sort
SELECT * FROM movies
WHERE title LIKE '%spider%'
ORDER BY year DESC;

-- Fix one row by ID
UPDATE movies SET rating = 9.5 WHERE id = 1;

-- Safe delete: find first, then delete by ID
SELECT id, title FROM movies WHERE title LIKE '%room%';
DELETE FROM movies WHERE id = 42;
```